STAGE 1 . BRONZE INGESTION . SCRUM-2

cache() vs persist() -- the double-read in ingest_dimensions.py

A walk through fixing the double CSV read in ingest_dimensions.py, and the cache-vs-persist question that came out of it -- with the official Spark documentation for each claim below.

FILE: src/bronze/ingest_dimensions.py LINES: 19, 26

01 The bug: two reads, one file

DataFrames are lazy. spark.read.schema(...).csv(...) doesn't touch disk -- it just builds a plan. The plan only runs when an action forces it.

```
dimensions_df = spark.read.schema(schema)...csv(...)
row_count = dimensions_df.count()        # action #1 -- reads the whole file
...
dimensions_df.write...csv(...)            # action #2 -- reads the whole file AGAIN
```

Nothing sits between the two actions to remember the first read, so Spark parses the CSV from scratch twice -- once for a log line, once to actually persist it.

02 First instinct: .cache(). But where?

The obvious fix is to cache the DataFrame so the second action reuses the first one's work. The question that actually matters is placement: does .cache() have to sit right before .count(), or does it work wherever it's called?

> **WHAT .CACHE() ACTUALLY IS**
> .cache() is not an action. It's a note left on the DataFrame: "the next time some action computes you, keep the result." It does nothing by itself.

03 Placement is mechanical, not stylistic

Because caching only happens on the next action, it has to be called before the first action in the code -- which here is .count(). Cache after .count() and you've already paid for one uncached scan; you'd still have two.

```
dimensions_df = spark.read.schema(schema)...csv(...)
dimensions_df.cache()                    # mark it -- no I/O yet
row_count = dimensions_df.count()        # this action reads AND fills the cache
...
dimensions_df.write...csv(...)            # reads from cache, not from the CSV
```

04 Then: is caching even the right tool?

Caching a full DataFrame in memory just to print a row count is a real cost for a small benefit. Worth naming the alternatives before defaulting to cache: (a) cache before .count(), keep the log line; (b) drop the pre-write count entirely, down to one action; (c) derive the count from the write job itself.

Assuming the log line is worth keeping -- which is where this recap picks back up -- the next question is which caching call to use.

05 cache() is persist() with the choice already made

.cache() and .persist() run the same machinery. The only difference is that .cache() takes no argument -- it always uses one fixed storage level. .persist(level) lets you name that level yourself.

The fixed level differs by type, and this is where it's easy to guess wrong:

- RDD.cache() -> MEMORY_ONLY
- DataFrame.cache() -> MEMORY_AND_DISK

This code works with a DataFrame, so .cache() here means memory first, spill to disk if it doesn't fit -- not RDD's memory-only behavior.

06 Why the memory-vs-disk distinction is not academic here

The Spark container in docker-compose.yml runs locally alongside MinIO with no explicit memory tuning. If a table's cached size didn't fit in whatever memory the executor actually has, and the storage level were MEMORY_ONLY, Spark wouldn't error -- it would silently drop the partitions that didn't fit and recompute them from source the next time they're touched.

Recompute from source means: re-read the CSV. That's the exact bug this whole exercise is fixing.

> **WHY MEMORY_AND_DISK IS THE SAFER DEFAULT**
>
> MEMORY_AND_DISK spills overflow to local disk instead of dropping it. Slower than RAM, but it never falls back to the original file -- so under memory pressure it degrades gracefully instead of quietly reintroducing the double-read.

Storage levels at a glance

STORAGE LEVEL	BEHAVIOR	WHEN IT FITS
`MEMORY_ONLY`	Deserialized objects in RAM. Overflow partitions are dropped and recomputed from source on next use.	Small dataset, known to fit, and re-reading the source is genuinely cheap.
`MEMORY_AND_DISK`	RAM first, spills overflow to local disk instead of dropping it. Default for Dataset.cache().	The general-purpose choice -- this project's bronze ingestion included.
`MEMORY_ONLY_SER`	Serialized bytes in RAM -- smaller footprint, extra CPU cost to deserialize on read.	Memory is tight but CPU headroom is available.
`DISK_ONLY`	Never touches RAM for storage. Protects JVM heap entirely.	Very large intermediate result you must not let pressure GC.
`MEMORY_AND_DISK_2`	Same as MEMORY_AND_DISK, replicated to two executors.	Reused across a long, fault-sensitive job -- not a fit for a one-shot ingestion script.

> **THE CALL FOR THIS FILE**
>
> Plain .cache() is enough here. It already resolves to MEMORY_AND_DISK for a DataFrame, which is exactly the "don't silently fall back to re-reading the CSV" behavior this fix needs -- no explicit storage level required.
>
> Reach for .persist(StorageLevel...) only when there's a concrete, known reason to override that default -- e.g. forcing DISK_ONLY on something large enough to threaten heap pressure, or MEMORY_ONLY once you've actually confirmed a table is small and disk-spill bookkeeping is pure overhead.

Further reading -- official Spark documentation

RDD Programming Guide -- RDD Persistence

The canoni

replication) a

https://spark.apache.org/docs/latest/rdd-programming-guide.html#rdd-persistence

pyspark.sql.DataFrame.cache

API referenc

https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.cache.html

pyspark.sql.DataFrame.persist

API referenc

https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.persist.html

pyspark.StorageLevel

The Storage
below is buil

https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.StorageLevel.html

Spark SQL Performance Tuning -- Caching Data In Memory

Spark-SQL-
directly in th

https://spark.apache.org/docs/latest/sql-performance-tuning.html#caching-data-in-memory